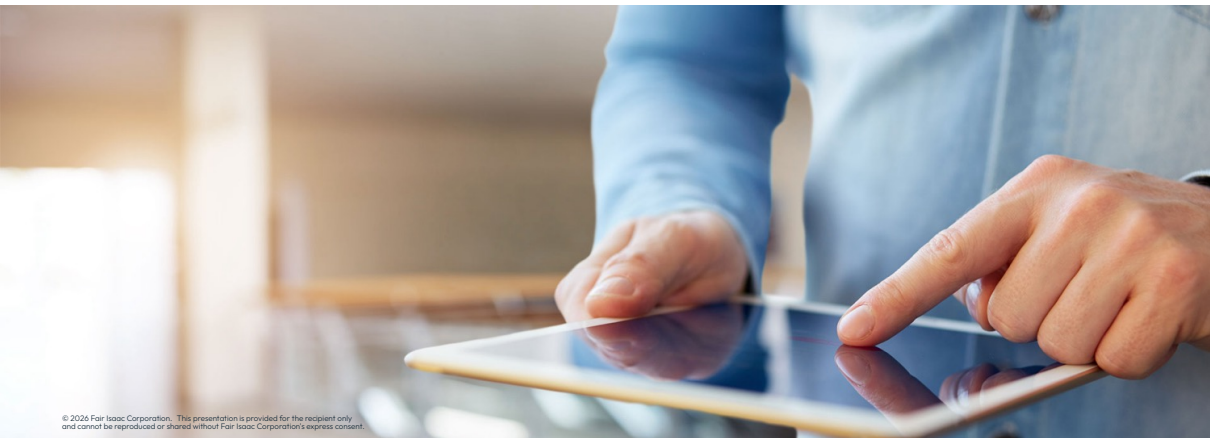


Controls and attributes



 Controls and attributes

Controls and attributes

- The Xpress Python interface enables the user to set controls and query attributes of a problem:
 - A **control** is a parameter that can **influence the behavior** (and therefore the performance) of FICO® Xpress Optimizer:
 - For example: the MIP gap target, the feasibility tolerance, or the type of root LP algorithms are controls that can be defined by the user
 - Problem controls **can both be read from and written to** an optimization problem

Controls and attributes

- The Xpress Python interface enables the user to set controls and query attributes of a problem:
 - A **control** is a parameter that can **influence the behavior** (and therefore the performance) of FICO® Xpress Optimizer:
 - For example: the MIP gap target, the feasibility tolerance, or the type of root LP algorithms are controls that can be defined by the user
 - Problem controls **can both be read from and written to** an optimization problem
 - An **attribute** is a feature of an optimization problem, such as the number of rows and columns or the number of quadratic elements in the objective function:
 - They are read-only parameters, i.e. their value cannot be directly modified by the user
 - Can be accessed in much the same manner as for the controls



Finding help: *For a full list of controls and attributes, explore the **Controls** and **Attributes** chapters of the FICO® Xpress Optimizer reference manual*

Accessing problem controls as object members

- Every problem has a `problem.controls` object that stores the controls related to the problem itself:

```
p.controls.<controlname>           # read problem control  
p.controls.<controlname> = <new value> # set problem control
```

- The functions `p.getControl()` and `p.setControl()` refer to this object

Accessing problem controls as object members

- Every problem has a `problem.controls` object that stores the controls related to the problem itself:

```
p.controls.<controlname>           # read problem control  
p.controls.<controlname> = <new value> # set problem control
```

- The functions `p.getControl()` and `p.setControl()` refer to this object
- Examples:

```
print(p.controls.feastol)           # print feasibility tolerance  
p.controls.presolve = 0             # disable presolve for this problem  
p1.controls.miprelstop = 10 * p2.controls.miprelstop # p1's miprelstop derived from p2
```



Note: Control values are double precision and can be of three types: integer, floating point, string

Heuristic emphasis control

- The `problem.controls.heuremphasis` control specifies an emphasis for the search w.r.t. `primal heuristics and other procedures`:

```
p.controls.heuremphasis = 1    # set heuremphasis to 1  
p.optimize()
```

- This control `affects the speed of convergence` of the primal-dual gap and can be assigned a value:
 - 1: applies the default strategy
 - 0: disables all heuristics
 - 1: focus on reducing the primal-dual gap in the early part of the search
 - 2: applies apply extremely aggressive search heuristics

Heuristic emphasis control

- The `problem.controls.heuremphasis` control specifies an emphasis for the search w.r.t. `primal heuristics and other procedures`:

```
p.controls.heuremphasis = 1    # set heuremphasis to 1  
p.optimize()
```

- This control `affects the speed of convergence` of the primal-dual gap and can be assigned a value:
 - 1: applies the default strategy
 - 0: disables all heuristics
 - 1: focus on reducing the primal-dual gap in the early part of the search
 - 2: applies apply extremely aggressive search heuristics
- Values 1 and 2 trigger many additional heuristic calls, aiming for reducing the gap at the beginning of the search, typically `at the expense of an increased time for proving optimality`



Finding help: *To learn more about the heuristics applied by the FICO® Xpress Optimizer during a MIP solve, explore [the reference manual](#)*

Optimizer built-in Tuner

- The FICO® Xpress Optimizer **Tuner** is a tool to automate the process of discovering better control parameter settings:
 - Systematically tests the problem against a range of **different combinations of control settings**
 - Can be applied to either a single problem instance or a small collection of problem instances
 - A single tuning run will typically involve solving each problem at least 100-200 times:
 - Can therefore **become computationally very expensive for large problems**

Optimizer built-in Tuner

- The FICO® Xpress Optimizer **Tuner** is a tool to automate the process of discovering better control parameter settings:
 - Systematically tests the problem against a range of **different combinations of control settings**
 - Can be applied to either a single problem instance or a small collection of problem instances
 - A single tuning run will typically involve solving each problem at least 100-200 times:
 - Can therefore **become computationally very expensive for large problems**
 - Examples of tuner-related controls and functions:

```
p.controls.tunermaxtime = 100      # set max time spent in tuning
p.controls.tunerthreads = 2       # set no. threads used by the tuner
p.tunerWriteMethod('default.xtm') # export tuner options onto an XTM
p.tunerReadMethod('default.xtm')  # read tuner options from a file
p.tune('g')                       # tune the problem as a MIP
p.optimize()                      # optimize the problem with best control settings found
```



Finding help: Check the *Xpress Optimizer tuning guide* to learn more about the built-in Tuner

Accessing global controls as object members

- The Xpress module also has a **controls object** containing all controls of the Xpress Optimizer:
 - A "prompt-friendly" way to read and set controls of the Xpress module is by using the members of **xpress.controls**:

```
xp.controls.<controlname>           # read control  
xp.controls.<controlname> = <new value> # set control
```

- Upon importing the Xpress module, these controls are **initialized at their default value**
 - When a new problem is created, its controls are copied from the global object

Accessing global controls as object members

- The Xpress module also has a **controls object** containing all controls of the Xpress Optimizer:
 - A "prompt-friendly" way to read and set controls of the Xpress module is by using the members of **xpress.controls**:

```
xp.controls.<controlname>           # read control  
xp.controls.<controlname> = <new value> # set control
```

- Upon importing the Xpress module, these controls are **initialized at their default value**
- When a new problem is created, its controls are copied from the global object
- Examples:

```
if xp.controls.presolve: ...      # check if presolve is on or off  
print(xp.controls.heuremphasis)  # print heuristic emphasis control value  
xp.controls.feastol = 1e-4        # set feasibility tolerance to 1e-4
```



Note: Global controls are maintained throughout while the Xpress module is loaded and do not refer to any specific problem

Accessing problem attributes as object members

- Every problem has its own **attributes object** that stores the attributes related to the problem itself:

```
p.attributes.<attributename> # read attribute
```

- Handled by its members **the same way as with controls**, with two exceptions:
 - There is no "global" attribute object, as a set of attributes only makes sense when associated with a problem
 - An attribute cannot be set, thus it **can only be accessed for reading**

Accessing problem attributes as object members

- Every problem has its own **attributes object** that stores the attributes related to the problem itself:

```
p.attributes.<attributename> # read attribute
```

- Handled by its members **the same way as with controls**, with two exceptions:
 - There is no "global" attribute object, as a set of attributes only makes sense when associated with a problem
 - An attribute cannot be set, thus it **can only be accessed for reading**
 - Examples:

```
print(p.attributes.nodedepth) # print node depth
number_infeas_sets = p.attributes.numiis # get irreducible infeasible sets
print("MIPtol:", p.attributes.miprelstop)*100, "%") # print mip tolerance as %
```



Keep in mind: Attributes are only available after a problem *p* has been created or read from a file



Thank You

www.fico.com/optimization